



CHAPTER 6

Other Binary Trees:
Expression Tree, Huffman Tree & Heap

BINARY EXPRESSION TREE



BINARY EXPRESSION TREE

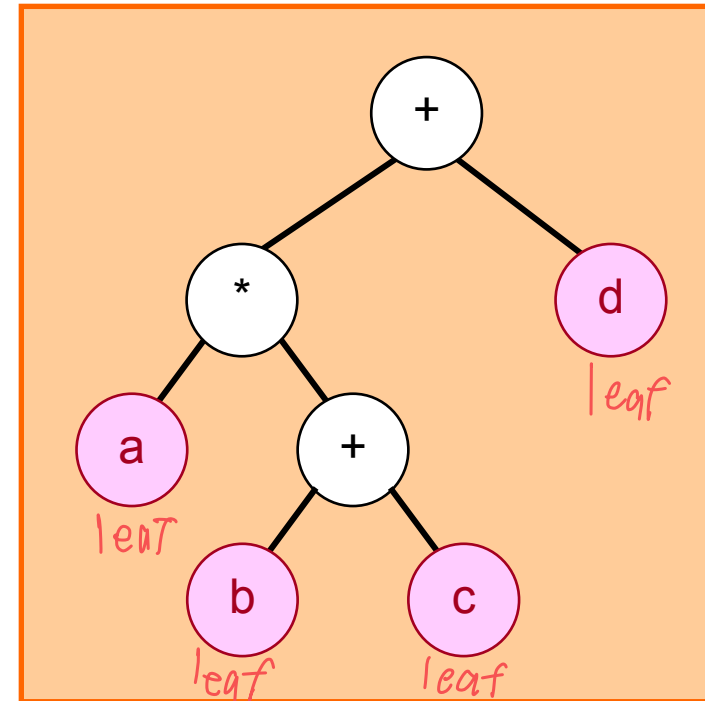
- Expression คือ นิพจน์การคำนวณซึ่งประกอบด้วย
 - Operator คือ ตัวดำเนินการ เช่น +, -, *, /
 - Operand คือ ตัวถูกดำเนินการ

Ex. $52 + 10 \rightarrow$ Operand : 52, 10 Operator : +

- Expression Tree : เป็น Binary tree ที่ใช้สำหรับจัดเก็บ Expression โดยมีลักษณะดังนี้
 - Leaf Node จะเป็น Operand เสมอ
 - Root และ Internal Node จะเป็น Operator
 - Subtree ที่มี root เป็น Operator จะเป็น subexpression

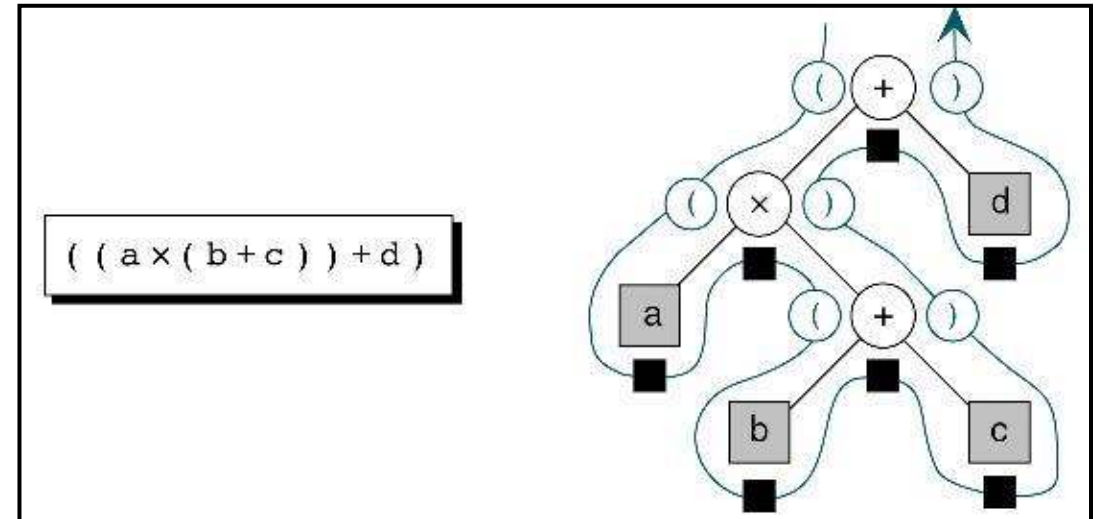
EXPRESSION TREE EXAMPLE

- $a*(b+c)+d$

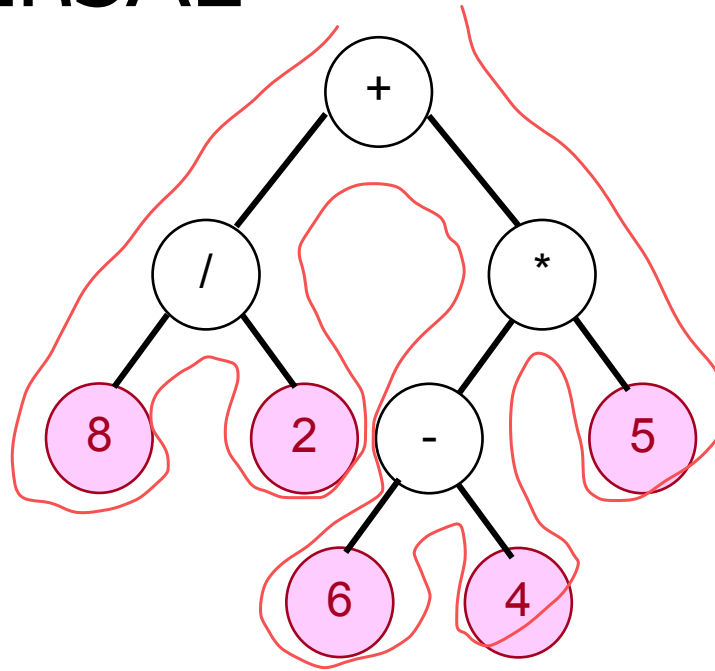


EXPRESSION TREE : TRAVERSAL

- ❖ ในกรณีของการทอแบบ Infix จะต้องมีการเพิ่มวงเล็บเปิดและปิดเมื่อเริ่มต้นและจบเอ็กซ์เพรสชัน ตามลำดับ
 - เปิดวงเล็บเมื่อเริ่มต้น tree หรือ subtree
 - ปิดวงเล็บเมื่อทำงาน tree หรือ subtree นั้นเสร็จ
- ❖ ในกรณีการทอแบบ Prefix และ Postfix ไม่จำเป็นต้องมีวงเล็บเปิดปิด



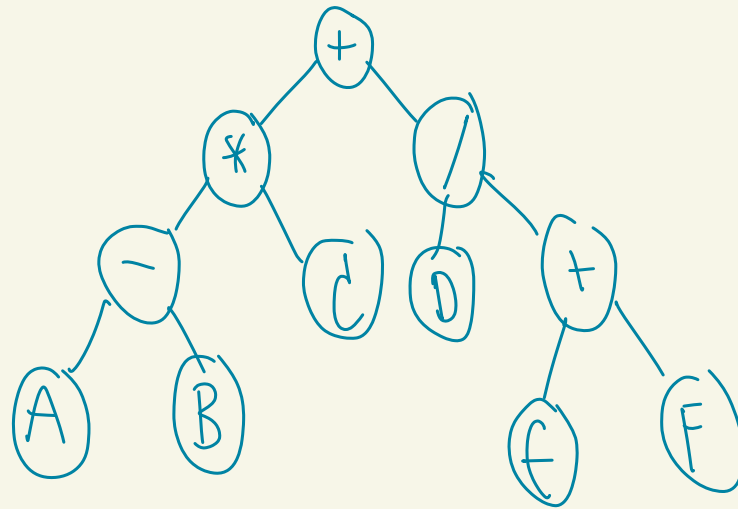
EXAMPLE : TRAVERSAL



- Preorder Traversal : $+ / 8 2 * - 6 4 5$ $\rightarrow$ Prefix
- Inorder Traversal : $((8 / 2) + ((6 - 4) * 5))$ $\rightarrow$ Infix
- Postorder Traversal : $8 2 / 6 4 - 5 * +$ $\rightarrow$ Postfix

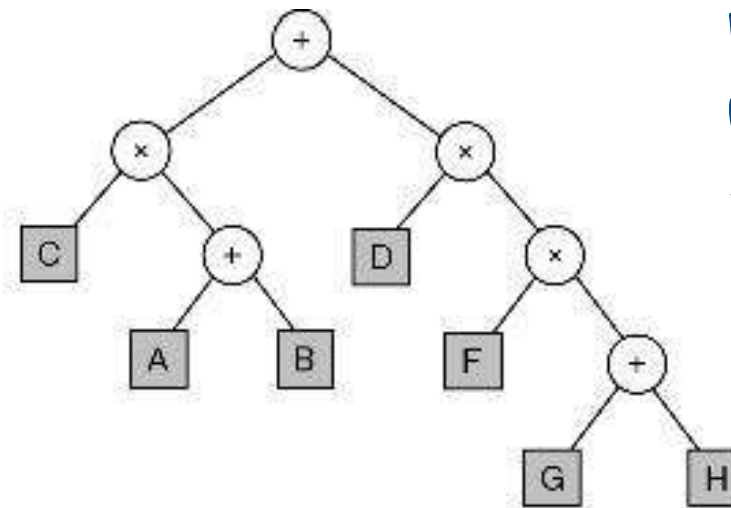
prefix : $+ * - A B C / D + E F$

Q10 Binary Expression Tree



EXERCISE – EXPRESSION TREE

- จาก Expression Tree ให้หา infix, prefix, postfix expression



infix $((C \times (A + B)) + (D \times (F \times (G + H))))$

pre $+ \times C + A B \times D \times F + G H$

post $C A B + \times D F G H + \times \times +$

- ให้เขียน Expression Tree, Infix, Postfix สำหรับ Prefix expression ดังนี้

$+ - * A + B C / D E * F G$

HUFFMAN TREE



HUFFMAN CODE

❖ เราใช้รหัส ASCII แทนอักขระต่างๆ ซึ่ง

- เข้ารหัสโดยใช้ 8 บิตต่อ 1 อักขระ เช่น อักขระ A มีรหัส ASCII เป็น 0100 0001 (65)
- เป็นการเข้ารหัสแบบความยาวคงที่ (Fixed Length) ถ้าข้อความประกอบด้วย 4 ตัวอักษร ก็ต้องใช้ $4 \times 8 = 32$ บิต เพื่อกำหนดข้อความนั้น

❖ Huffman Code :

- เป็นการเข้ารหัสแบบความยาวไม่คงที่ (Variable Length)
- ทำการเข้ารหัสโดยพิจารณาจากความถี่ของอักขระที่ปรากฏในเอกสาร
- ทำให้ใช้เนื้อที่ในการเก็บข้อมูลน้อยลง

EXAMPLE : HUFFMAN CODE

- ถ้ามีข้อความ A B C A D B A E E B A
 - เข้ารหัส ASCII จะได้ 01000001 01000010 (11*8 = 88 บิต)
 - เข้ารหัส Huffman จะพิจารณาจากความถี่ของอักขระที่ปรากฏ นั่นคือ A, B, C, D, E มีความถี่ 4, 3, 1, 1, 2 ตามลำดับ
 - อักขระที่มีความถี่มาก จะใช้รหัสสั้นกว่า อักขระที่มีความถี่น้อย ซึ่งกำหนดให้แทน
 - A มีรหัส 11
 - B มีรหัส 10
 - C มีรหัส 000
 - D มีรหัส 001
 - E มีรหัส 01
 - จากโจทย์เมื่อเข้ารหัสจะได้ 11 10 000 11 001 10 11 01 01 10 11 (24 บิต)

HUFFMAN TREE

ปัญหา แล้วจะแทนอักขระอย่างไร (รู้ได้อย่างไรว่าแทน A เป็น 11)

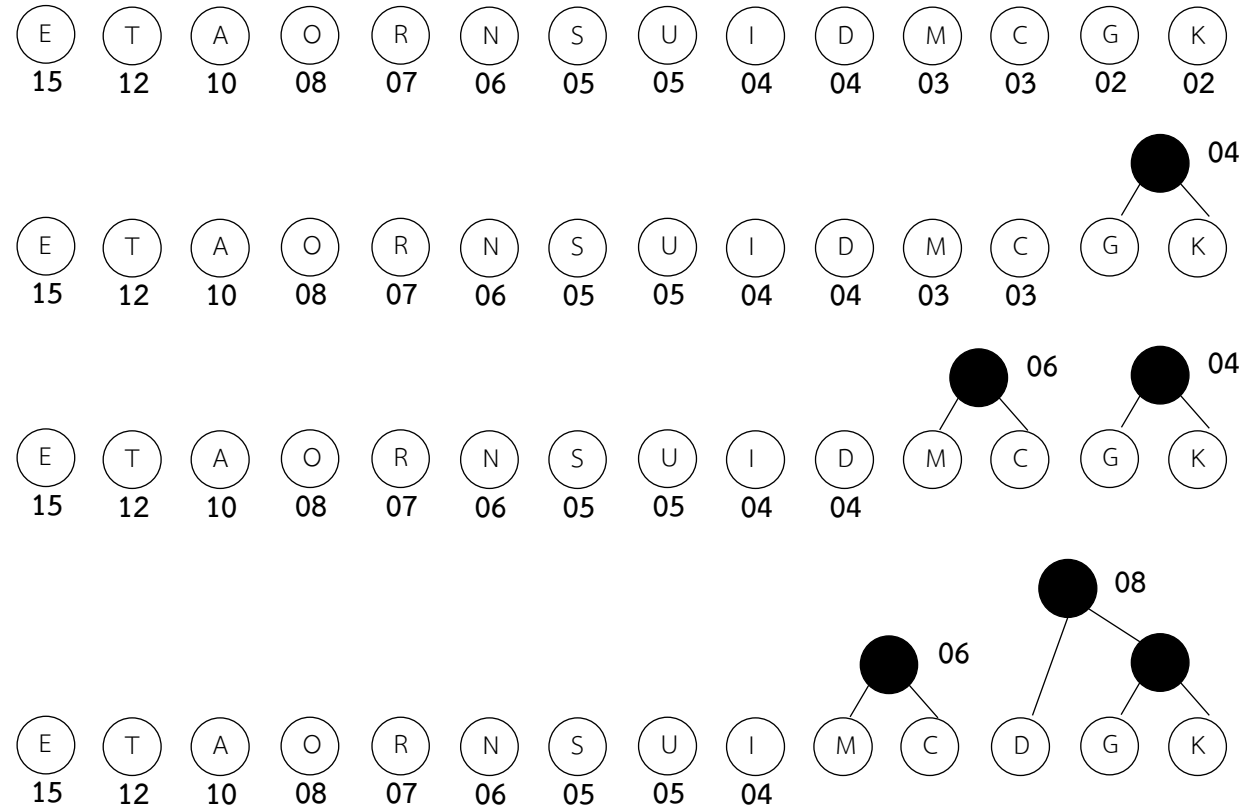
ตอบ ใช้ Huffman Tree ซึ่งสามารถสร้างได้ตามขั้นตอนดังนี้

- เรียงอักขระตามลำดับความถี่จากน้อยไปมาก และมองอักขระเป็นโหนด
- จับโหนดที่มีความถี่น้อยที่สุด 2 โหนด ให้สร้างต้นไม้ย่อย โดยมีโหนดพ่อแม่เดียวกัน
 - ความถี่โหนดพ่อแม่ เท่ากับ ผลบวกของความถี่ของโหนดลูก
- ทำไปเรื่อยๆ จนได้ต้นไม้ที่ประกอบด้วยโหนดอักขระครบทั้งหมด
- กำหนดโค้ดสำหรับอักขระแต่ละตัว
 - เขียนค่าเข้ารหัสบนกิ่งซ้ายและขวาของต้นไม้ ให้มีค่า 0 และ 1 ตามลำดับ
 - โค้ดของอักขระ : ลำดับเลขรหัสที่ไล่จากรูทจนถึงโหนดอักขระนั้นๆ

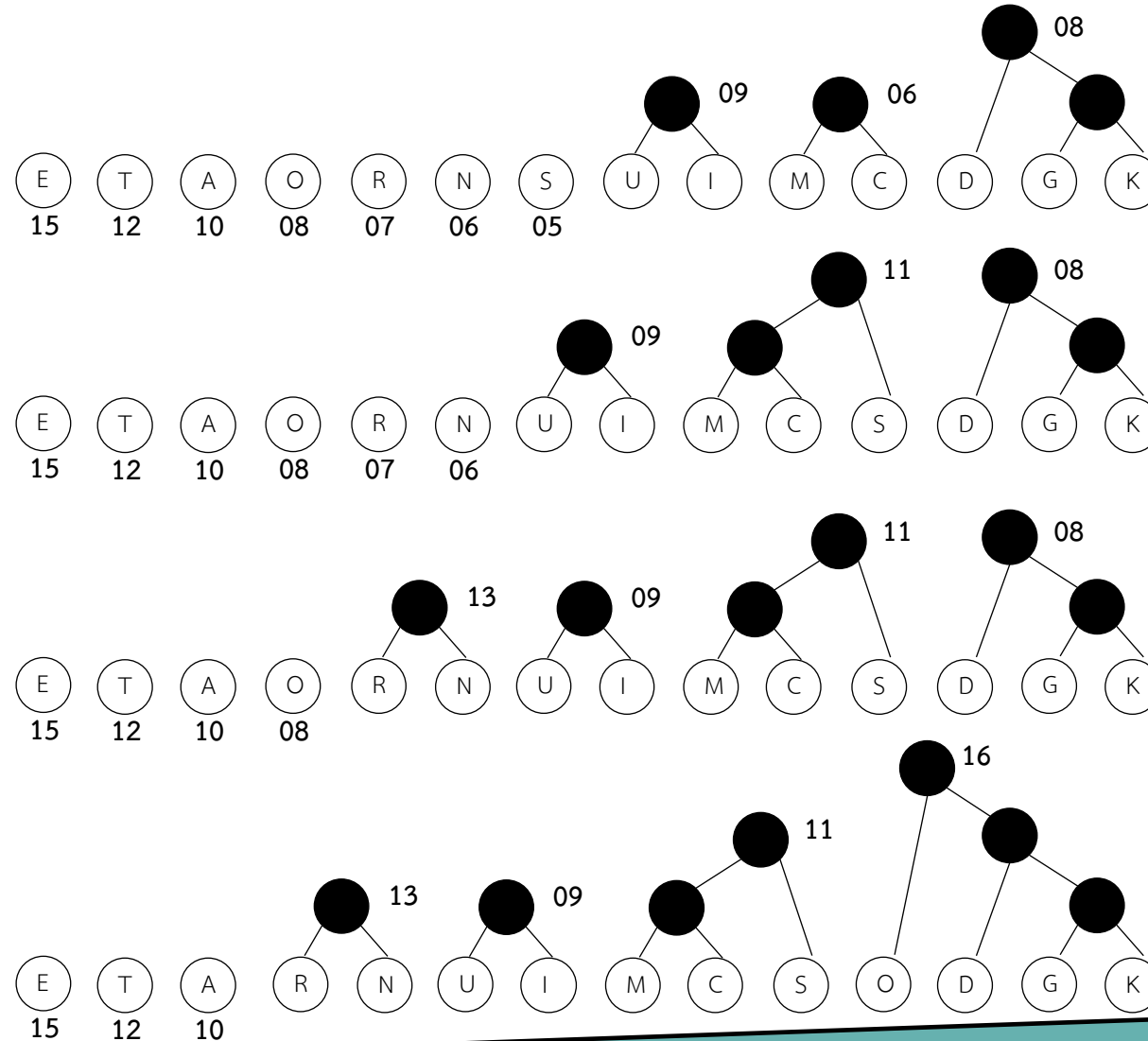
HUFFMAN TREE (CONT.)

Character	Weight	Character	Weight	Character	Weight
A	10	I	4	R	7
C	3	K	2	S	5
D	4	M	3	T	12
E	15	N	6	U	5
G	2	O	8		

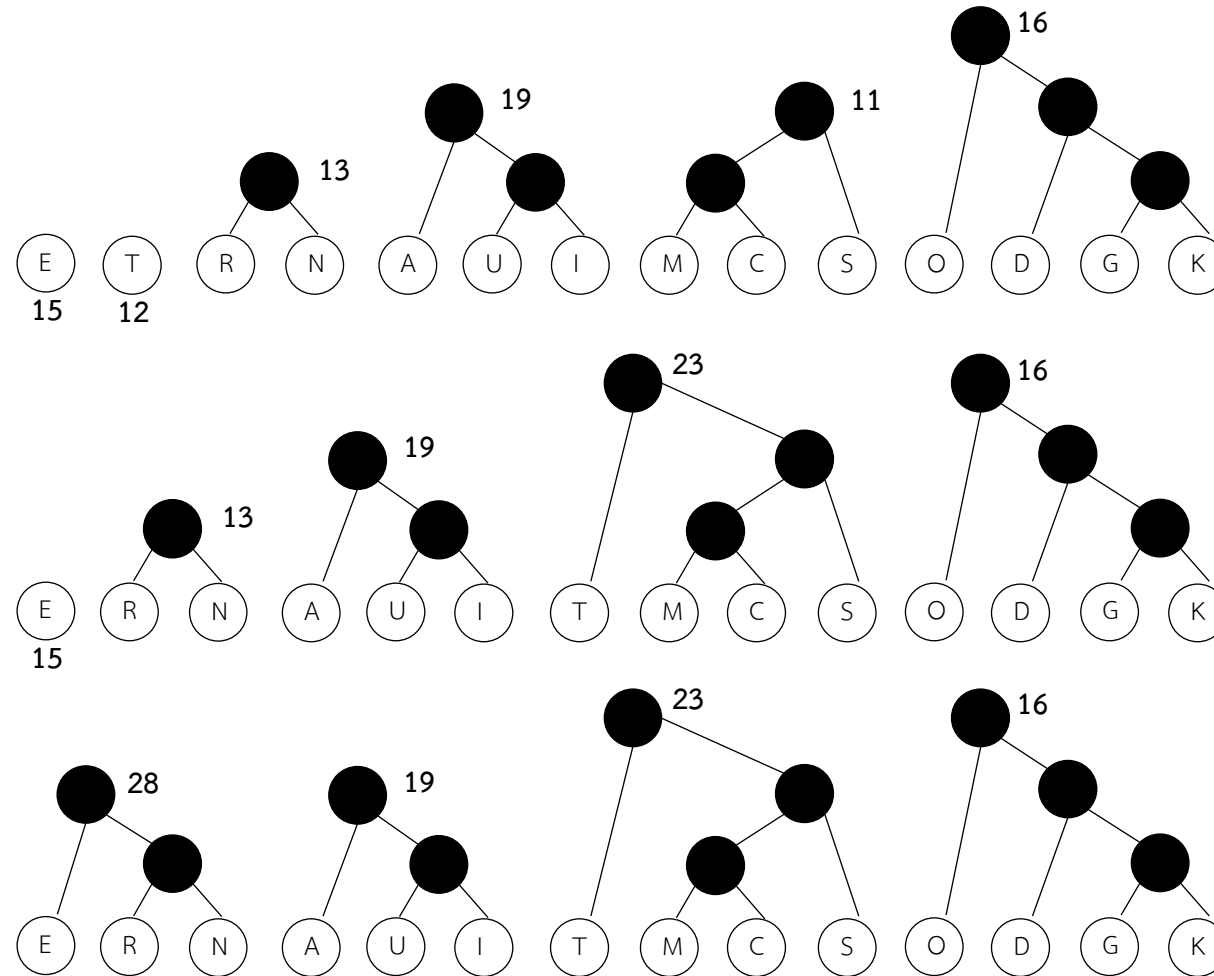
HUFFMAN TREE (CONT.)



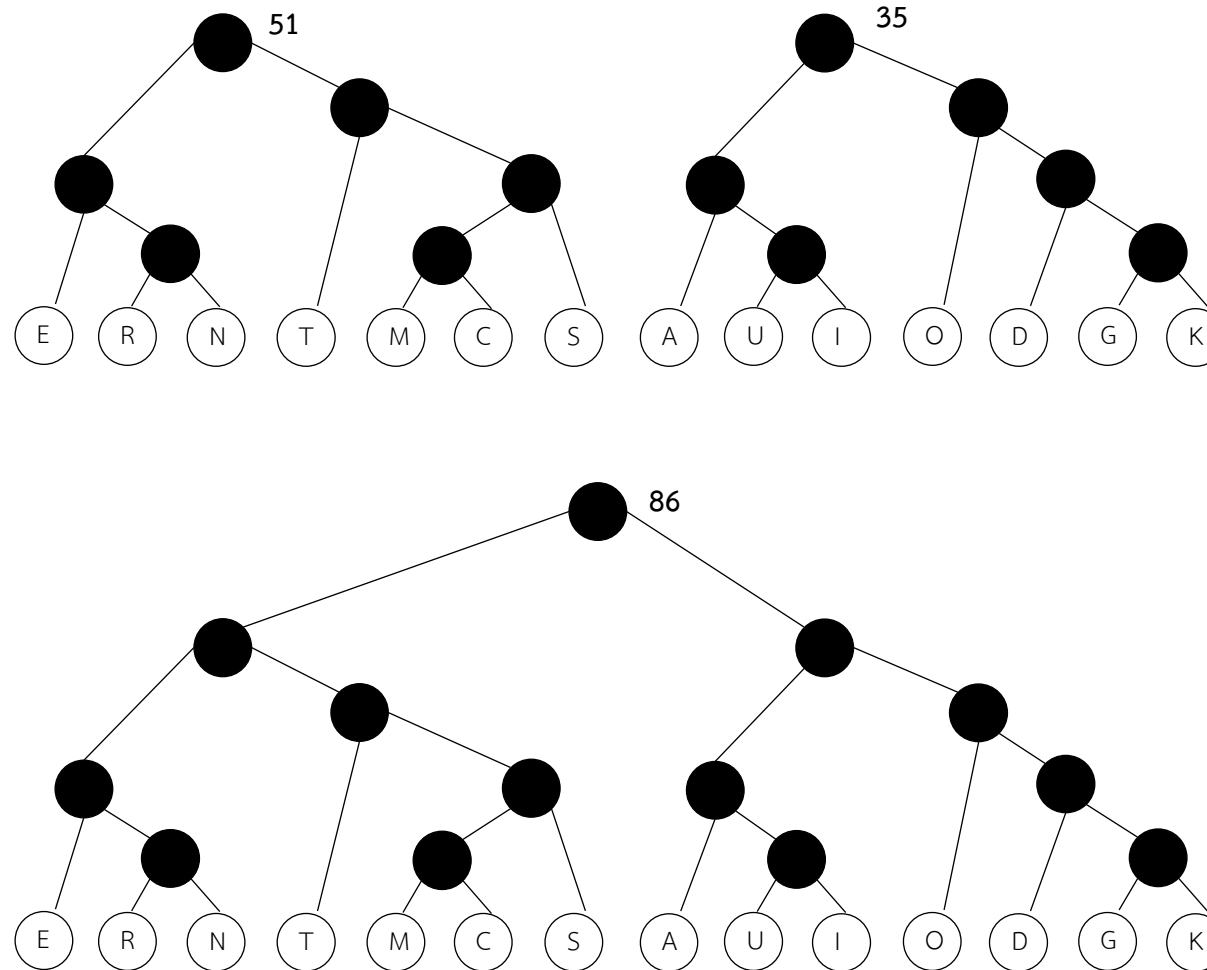
HUFFMAN TREE (CONT.)



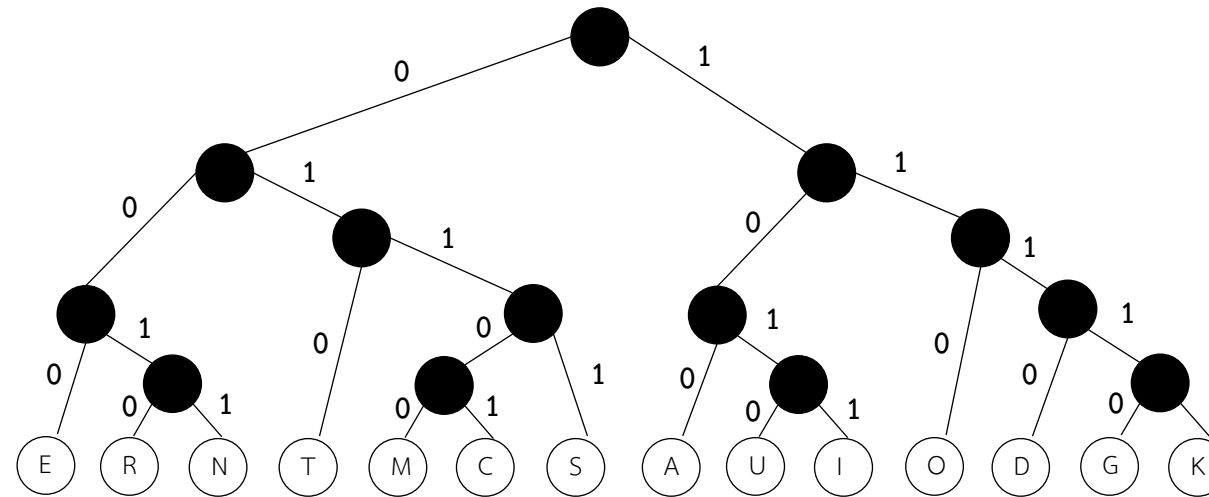
HUFFMAN TREE (CONT.)



HUFFMAN TREE (CONT.)



HUFFMAN TREE (CONT.)



E=000 R=0010 N=0011 T=010 M=01100 C=01101
S=0111 A=100 U=1010 I=1011 O=110 D=1110
G=11110 K=11111

EXERCISE – HUFFMAN TREE

จงสร้าง Huffman Tree จากตารางอักขระและความถี่ และ
ตอบคำถามต่อไปนี้

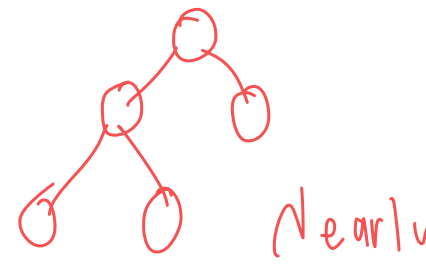
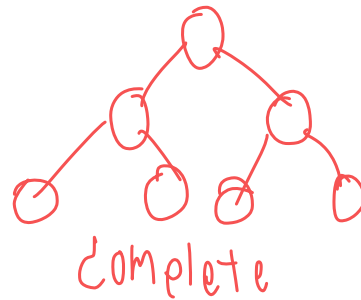
- จงสรุป Huffman code ของแต่ละตัวอักษร
- หากเอกสารประกอบด้วยตัวอักษรและมีจำนวนครั้งที่ปรากฏในเอกสารตามตาราง จงคำนวณขนาดของเอกสารในกรณีที่
 - เข้ารหัสตัวอักษรด้วย ASCII
 - เข้ารหัสตัวอักษรด้วย Huffman Code

อักษร	ความถี่
A	15
B	6
C	7
D	12
E	25
F	4

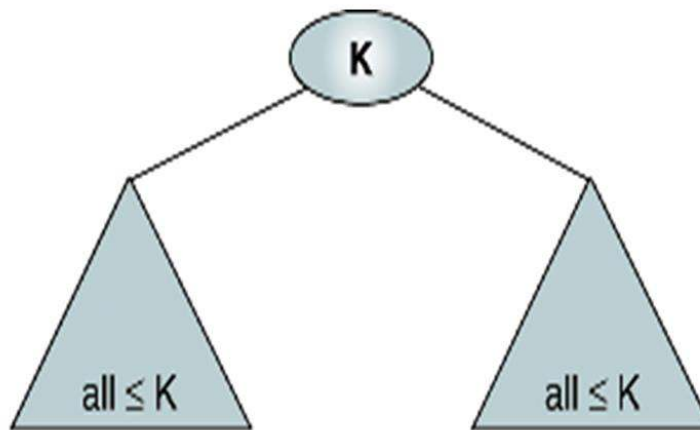
HEAP



HEAP (MAX-HEAP)

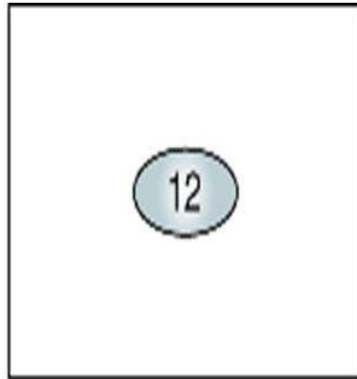


- เป็น Complete binary tree หรือ Nearly complete binary tree (Rule 1)
- โหนดทั้งหมดในต้นไม้ย่อยฝั่งซ้าย (Left subtree) และต้นไม้ย่อยฝั่งขวา (Right subtree) มีค่าน้อยกว่าหรือเท่ากับโหนดพ่อ (Rule 2)

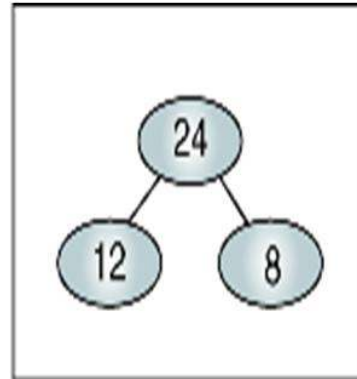


- Min-Heap: โหนดทั้งหมดในต้นไม้ย่อยฝั่งซ้าย (Left subtree) และต้นไม้ย่อยฝั่งขวา (Right subtree) มีค่ามากกว่าหรือเท่ากับโหนดพ่อ

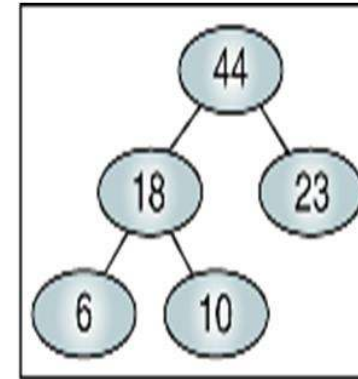
HEAP TREES



(a) Root-only heap

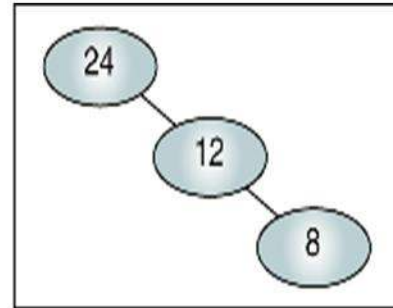


(b) Two-level heap

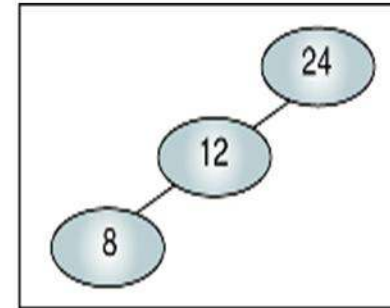


(c) Three-level heap

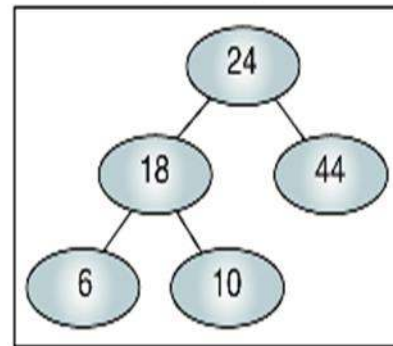
INVALID HEAPS



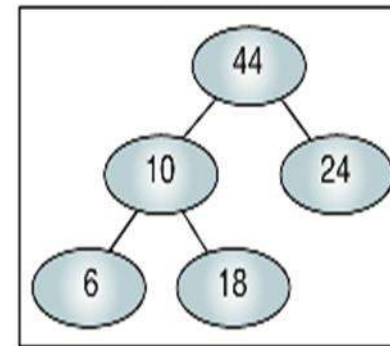
(a) Not nearly complete
(rule 1)



(b) Not nearly complete
(rule 1)



(c) Root not largest
(rule 2)

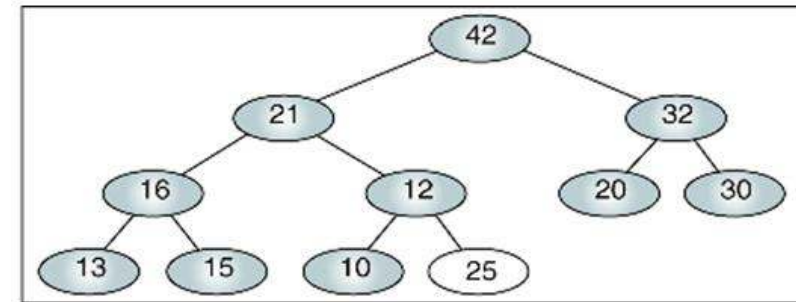
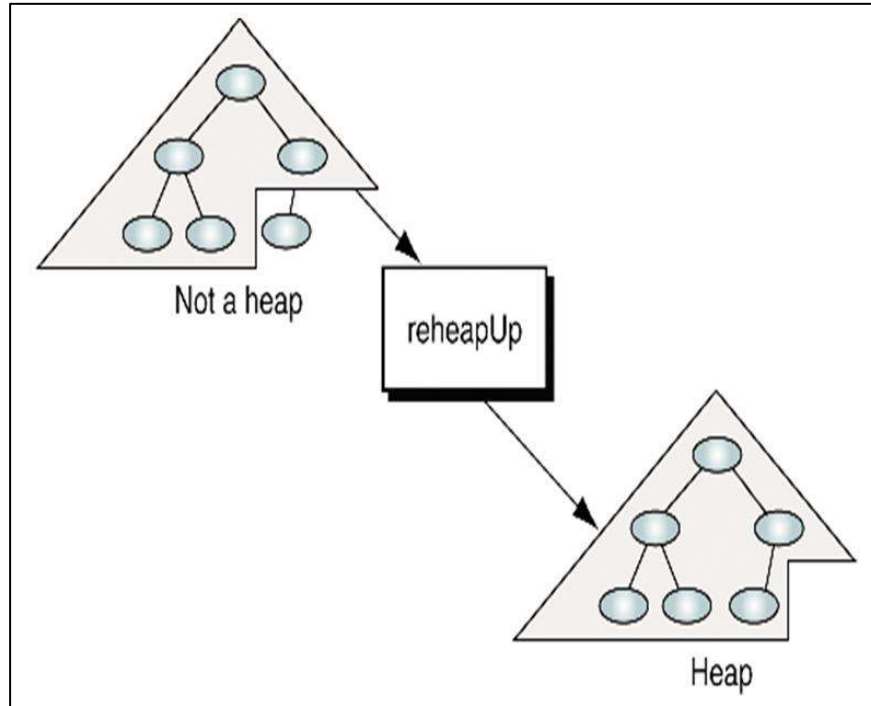


(d) Subtree 10 not a heap
(rule 2)

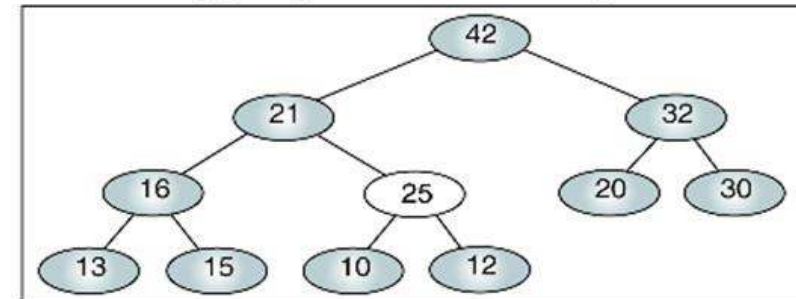
MAINTENANCE OPERATIONS

- การทำงานเบื้องต้นเพื่อใช้ในการแทรก (Insert) หรือ ลบ (Delete) โหนดออกจาก Heap
- Reheap Up → Insert เข้าด้านล่าง
 - ปรับโครงสร้าง Heap จากล่างขึ้นบน
- Reheap Down → Delete ออกจากบน
 - ปรับโครงสร้าง Heap จากบนลงล่าง

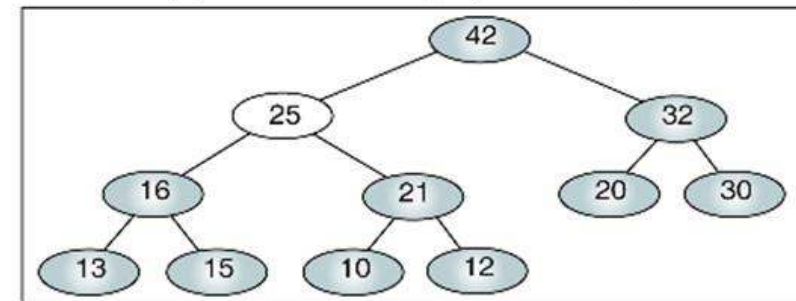
REHEAP UP



(a) Original tree: not a heap



(b) Last element (25) moved up

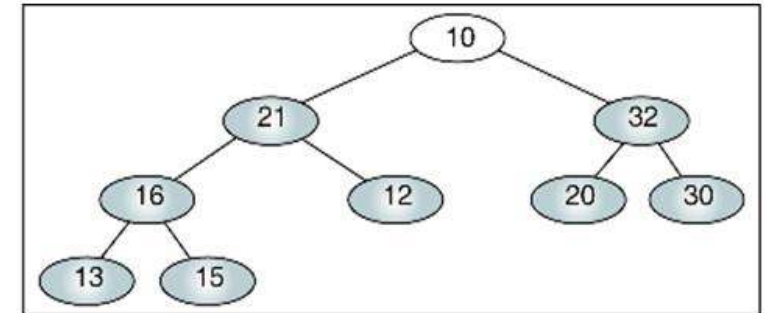
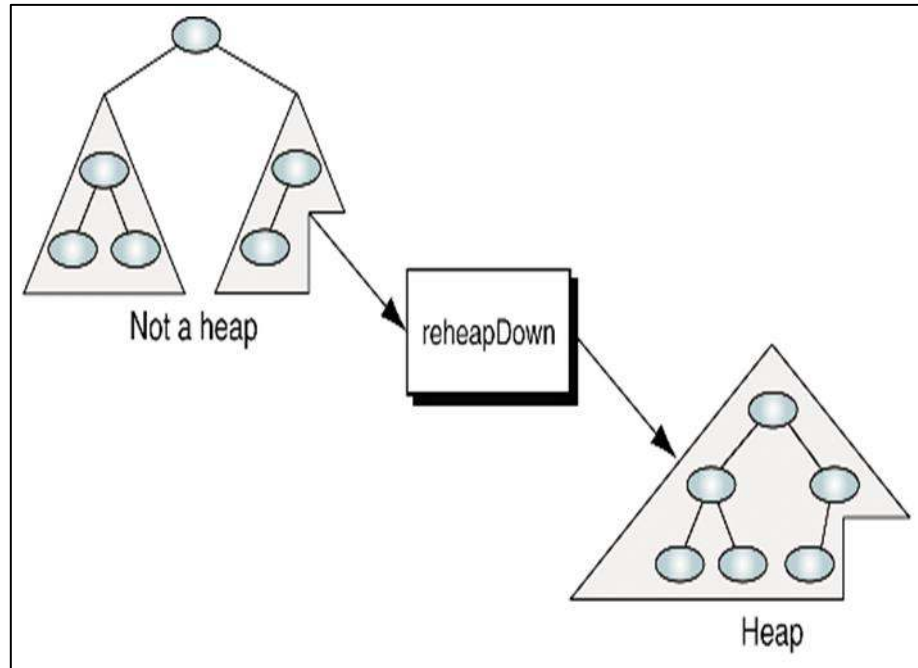


(c) Moved up again: tree is a heap

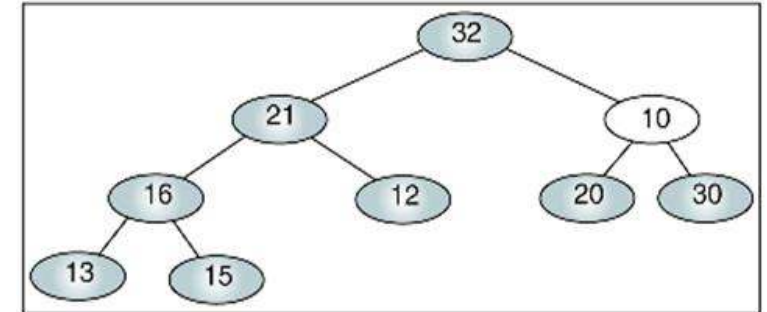
REHEAP UP

```
Algorithm reheapUp (heap, newNode)
Reestablishes heap by moving data in child up to its
correct location in the heap array.
  Pre  heap is array containing an invalid heap
       ¬newNode is index location to new data in heap
  Post heap has been reordered
1 if (newNode not the root)
  1  set parent to parent of newNode
  2  if (newNode key > parent key)
    1  exchange newNode and parent)
    2  reheapUp (heap, parent)
  3  end if
2 end if
end reheapUp
```

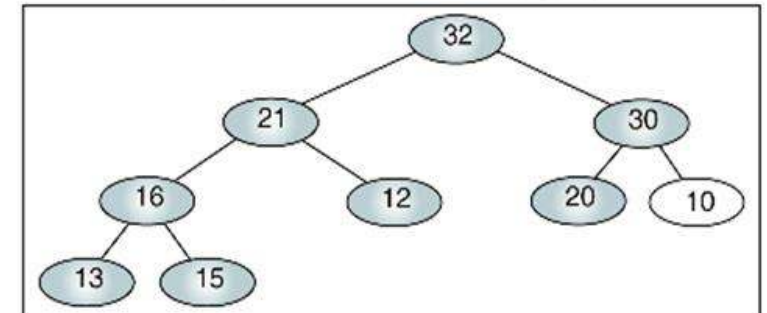
REHEAP DOWN



(a) Original tree: not a heap



(b) Root moved down (right)



(c) Moved down again: tree is a heap

REHEAP DOWN

Algorithm reheapDown (heap, root, last)

Reestablishes heap by moving data in root down to its correct location in the heap.

Pre heap is an array of data

 root is root of heap or subheap

 last is an index to the last element in heap

Post heap has been restored

Determine which child has larger key

1 if (there is a left subtree)

 1 set leftKey to left subtree key

 2 if (there is a right subtree)

 1 set rightKey to right subtree key

 3 else

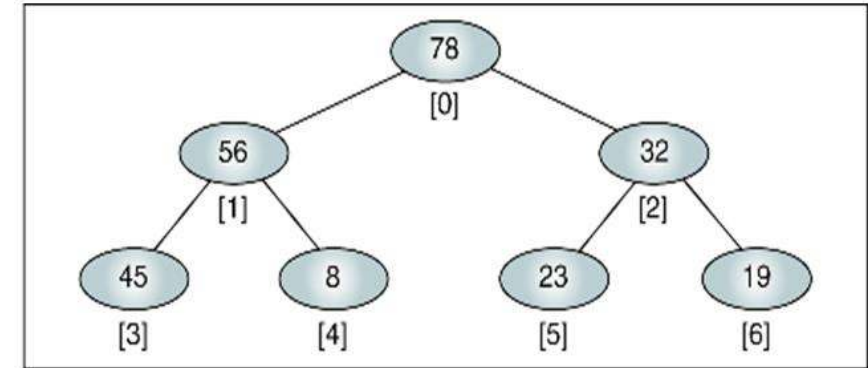
 1 set rightKey to null key

REHEAP DOWN

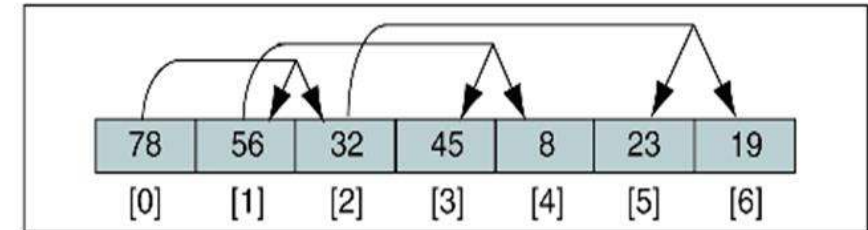
```
4  end if
5  if (leftKey > rightKey)
    1  set largeSubtree to left subtree
6  else
    1  set largeSubtree to right subtree
7  end if
  Test if root > larger subtree
8  if (root key < largeSubtree key)
    1  exchange root and largeSubtree
    2  reheapDown (heap, largeSubtree, last)
9  end if
2  end if
end reheapDown
```

HEAP IN ARRAY

- ปกติแล้ว มักจะใช้ Array ในการสร้าง Heap
- เนื่องจาก Heap เป็น Complete หรือ Nearly complete จึงสามารถหาความสัมพันธ์ระหว่างโหนดได้
- โหนดที่ตำแหน่ง i (ใน Array)
 - Left child: $2i+1$
 - Right child: $2i+2$
 - Parent: $\lfloor (i-1)/2 \rfloor$



(a) Heap in its logical form



(b) Heap in an array

HEAP IMPLEMENTATION

- Build a heap
- Insert a node into a heap
- Delete a node from a heap

BUILD A HEAP

- สร้าง Heap ได้ 2 แบบ คือ
 - 1) สร้าง Heap จากชุดข้อมูลที่มีอยู่แล้ว
 - 2) สร้าง Heap จากที่ไม่มีข้อมูลอยู่เลย
- ในกรณีที่มีชุดข้อมูลใน Array อยู่แล้ว (แต่ยังไม่มี การเรียงข้อมูลแบบ Heap)
 - สามารถสร้าง Heap ได้โดยการทำให้ Reheap up

Algorithm buildHeap (heap, size)

Given an array, rearrange data so that they form a heap.

Pre heap is array containing data in nonheap order

size is number of elements in array

Post array is now a heap

1 set walker to 1

2 loop (walker < size)

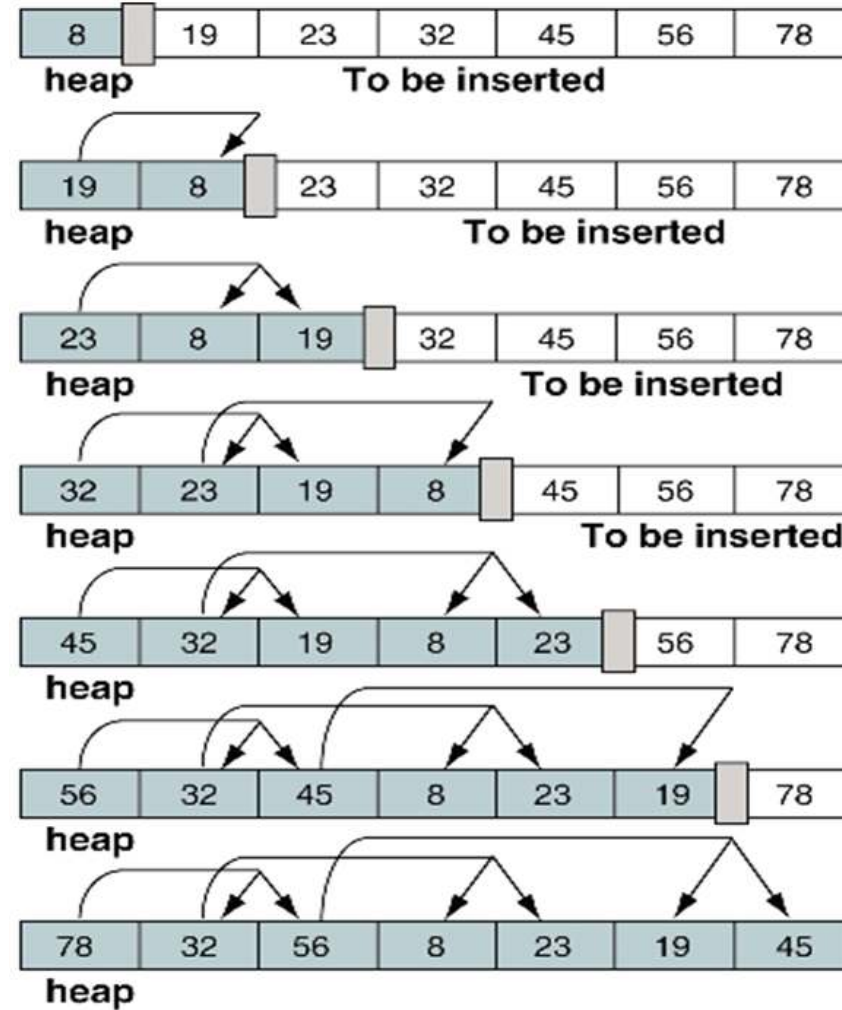
1 reheapUp(heap, walker)

2 increment walker

3 end loop

end buildHeap

BUILD A HEAP



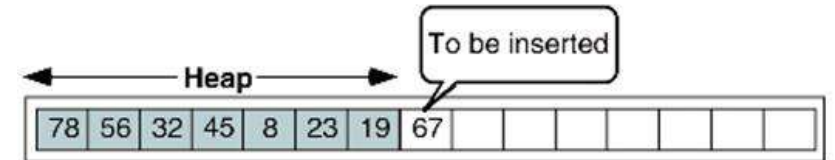
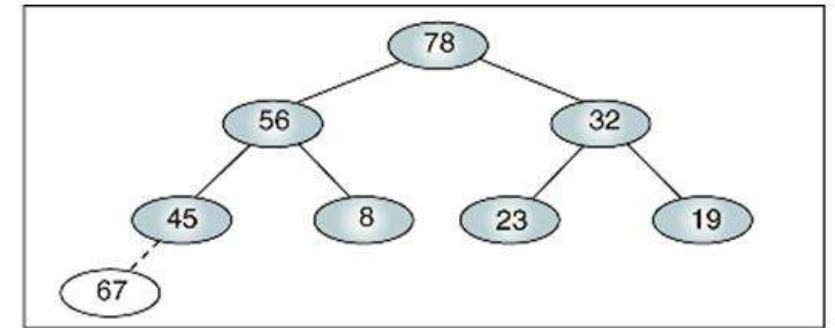
INSERT A NODE INTO A HEAP

- ทำการแทรกโหนดใหม่ที่ตำแหน่งท้ายสุดของ Heap แล้วทำการ Reheap up

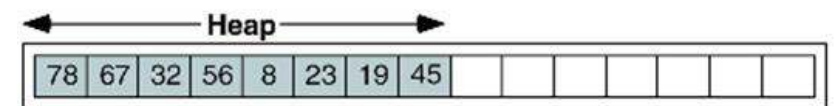
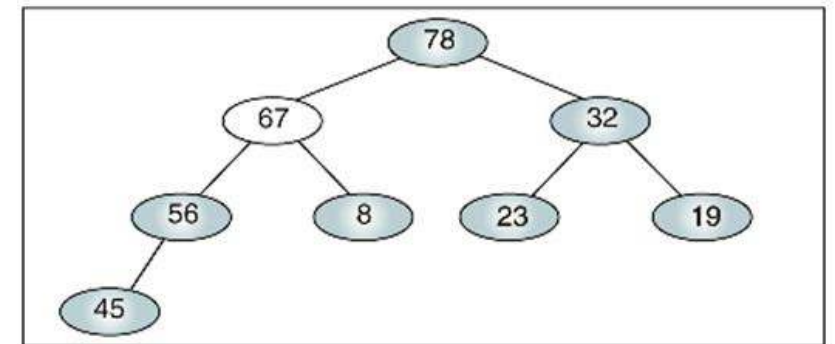
```
Algorithm insertHeap (heap, last, data)
Inserts data into heap.
  Pre    heap is a valid heap structure
         last is reference parameter to last node in heap
         data contains data to be inserted
  Post   data have been inserted into heap
  Return true if successful; false if array full
1 if (heap full)
  1 return false
2 end if
3 increment last
4 move data to last node
5 reheapUp (heap, last)
6 return true
end insertHeap
```

INSERT A NODE INTO A HEAP

- การสร้าง Heap โดยไม่มีข้อมูลเริ่มต้นเลย (Empty heap) ทำได้โดยการ Insert โหนด แล้ว Reheap up ไปเรื่อยๆ
- กรณีที่มี Heap อยู่แล้ว ให้ทำการแทรกโหนดที่ตำแหน่งสุดท้าย แล้วทำการ Reheap up



(a) Before reheap up



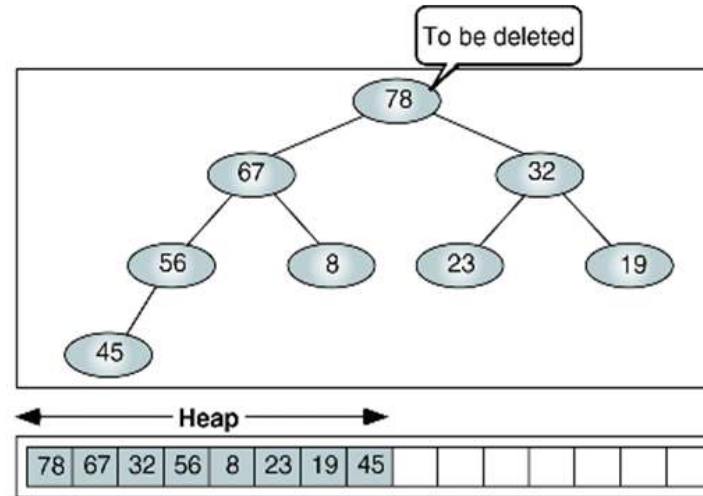
(b) After reheap up

DELETE A NODE FROM A HEAP

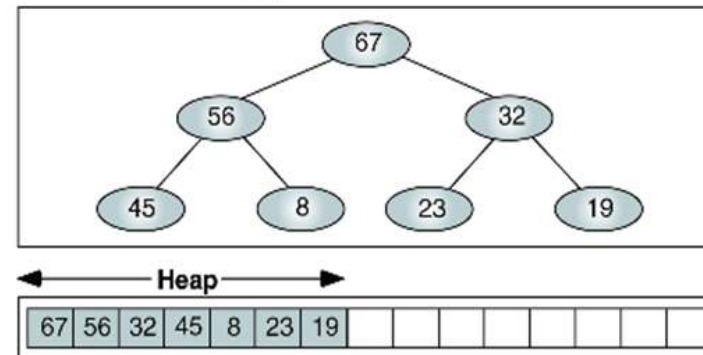
- ทำการลบ Root ออกจาก Heap โดยย้ายข้อมูลของโหนดสุดท้ายใน Heap มาแทนที่ Root แล้ว Reheap down

```
Algorithm deleteHeap (heap, last, dataOut)
Deletes root of heap and passes data back to caller.
Pre    heap is a valid heap structure
        last is reference parameter to last node in heap
        dataOut is reference parameter for output area
Post   root deleted and heap rebuilt
        root data placed in dataOut
Return true if successful; false if array empty
1 if (heap empty)
1   return false
2 end if
3 set dataOut to root data
4 move last data to root
5 decrement last
6 reheapDown (heap, 0, last)
7 return true
end deleteHeap
```

DELETE A NODE FROM A HEAP



(a) Before delete



(b) After delete

EXERCISE - HEAP

- จงแสดงขั้นตอนการสร้าง Heap เมื่อรับข้อมูลจากคีย์บอร์ดตามลำดับดังนี้ (เริ่มต้นเป็น Empty heap)
23 7 92 6 12 14 40 44 20 21
- จงวาด Heap ผลลัพธ์ เมื่อทำการลบโหนดจาก Heap นี้

